```
"""Run inference with a trained CIFAR-10 CNN checkpoint.

Usage:
    python inference.py path/to/image.jpg --model best_cifar10_model.pth
"""
import argparse
from pathlib import Path
import torch
import torch.nn as nn
from PIL import Image
from torchvision import transforms

class CIFAR10CNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(2), nn.Dropout(0.10),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, 3, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(2), nn.Dropout(0.15),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(inplace=True), nn.Dropout(0.30), nn.Linear(256, num_classes)
        )
    def forward(self, x):
        return self.classifier(self.features(x))

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("image", help="Path to an image file")
    parser.add_argument("--model", default="best_cifar10_model.pth", help="Check
```

```
point path")
    args = parser.parse_args()
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    checkpoint = torch.load(args.model, map_location=device, weights_only=False
)
    class_names = checkpoint.get("class_names", [
        "airplane", "automobile", "bird", "cat", "deer",
        "dog", "frog", "horse", "ship", "truck"])
    mean = checkpoint.get("mean", (0.4914, 0.4822, 0.4465))
    std = checkpoint.get("std", (0.2470, 0.2435, 0.2616))
    model = CIFAR10CNN(num_classes=len(class_names)).to(device)
    model.load_state_dict(checkpoint["model_state_dict"])
    model.eval()
    transform = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),
        transforms.Normalize(mean, std),
    ])
    image = Image.open(Path(args.image)).convert("RGB")
    tensor = transform(image).unsqueeze(0).to(device)
    with torch.no_grad():
        probs = torch.softmax(model(tensor), dim=1)[0]
    confidence, index = probs.max(dim=0)
    print(f"Predicted class: {class_names[index.item()]}")
    print(f"Confidence: {confidence.item():.2%}")
    print("Note: This model was trained on tiny CIFAR-10 images; arbitrary photos
may be out of distribution.")

if __name__ == "__main__":
    main()
```